

Aula 14 - Consultas Complexas: Filtragem, Agrupamentos e Junções (JOINS)

Descrição: Domínio da instrução `SELECT` para extração de relatórios estratégicos e cruzamento de dados relacionais utilizando cláusulas `WHERE`, `ORDER BY`, funções de agregação, `GROUP BY`, `HAVING` e a poderosa operação de `JOIN`.

DQL - Um caso à parte

A. CONCEITO

A **Linguagem de Consulta de Dados** é a parte do SQL focada em **extrair e visualizar** informações. É aqui que o banco de dados mostra seu verdadeiro valor: a capacidade de encontrar uma agulha no palheiro em milissegundos. O DQL não altera absolutamente nada no seu banco. Ele é como uma **lanterna**: você ilumina os dados para enxergá-los, mas a luz não muda a cor ou o formato do que está sendo iluminado.

O `SELECT` é, sem dúvida, o comando mais utilizado em todo o universo SQL. Ele funciona como um filtro inteligente que responde às suas perguntas. E como ele funciona na prática?

Imagine que você está diante do arquivo de aço da biblioteca e faz um pedido ao arquivista:

"Por favor, me mostre apenas o **Título** e o **Preço** de todos os livros que custam mais de **R\$ 50,00**."

No mundo do SQL, esse pedido "traduzido" seria:

1. **SELECT** (O que eu quero ver?): Título, Preço
2. **FROM** (Onde isso está guardado?): `LIVRO`
3. **WHERE** (Qual é o critério?): `Preço > 50.00`

B. VISUALIZANDO

ANALOGIA DO RESTAURANTE: ENTENDENDO **DML** vs. **DQL**

DML: MUDANDO O ESTADO
(Ação e Modificação)

O diagrama ilustra ações de DML em um restaurante. No topo, um garçom aponta para uma tabela com o botão **INSERT**. Abaixo, dois garçons fazem alterações: um cancela um pedido de pizza (botão **UPDATE**) e outro aumenta o preço de uma pizza de \$20 para \$25 (botão **UPDATE**). No rodapé, um texto explica: "O gerente altera dados, toma ações que mudam o estado (SENTAR CLIENTE, MUDAR PREÇO)." e uma seta vermelha rotulada **ALTERA DADOS** aponta para um ícone de banco de dados.

DQL (SELECT): CONSULTANDO INFORMAÇÃO
(Visualização e Leitura)

O diagrama ilustra ações de DQL em um restaurante. No topo, um cliente lê o cardápio (botão **CLIENTE LENDO**). Abaixo, um cliente lê o cardápio e um gerente consulta uma lista de pedidos no computador (botão **CONFERINDO PEDIDOS**). A lista de pedidos mostra: PIZZA \$230, PIZZA \$220, PIZZA \$225, e VENDAS DO DIA. No rodapé, um texto explica: "O cliente lê o cardápio ou o gerente vê a lista. APENAS CONSULTAM SEM ALTERAR NADA." e uma seta azul rotulada **LÊ DADOS** aponta para um ícone de banco de dados.

DML MUDA, DQL VÊ.

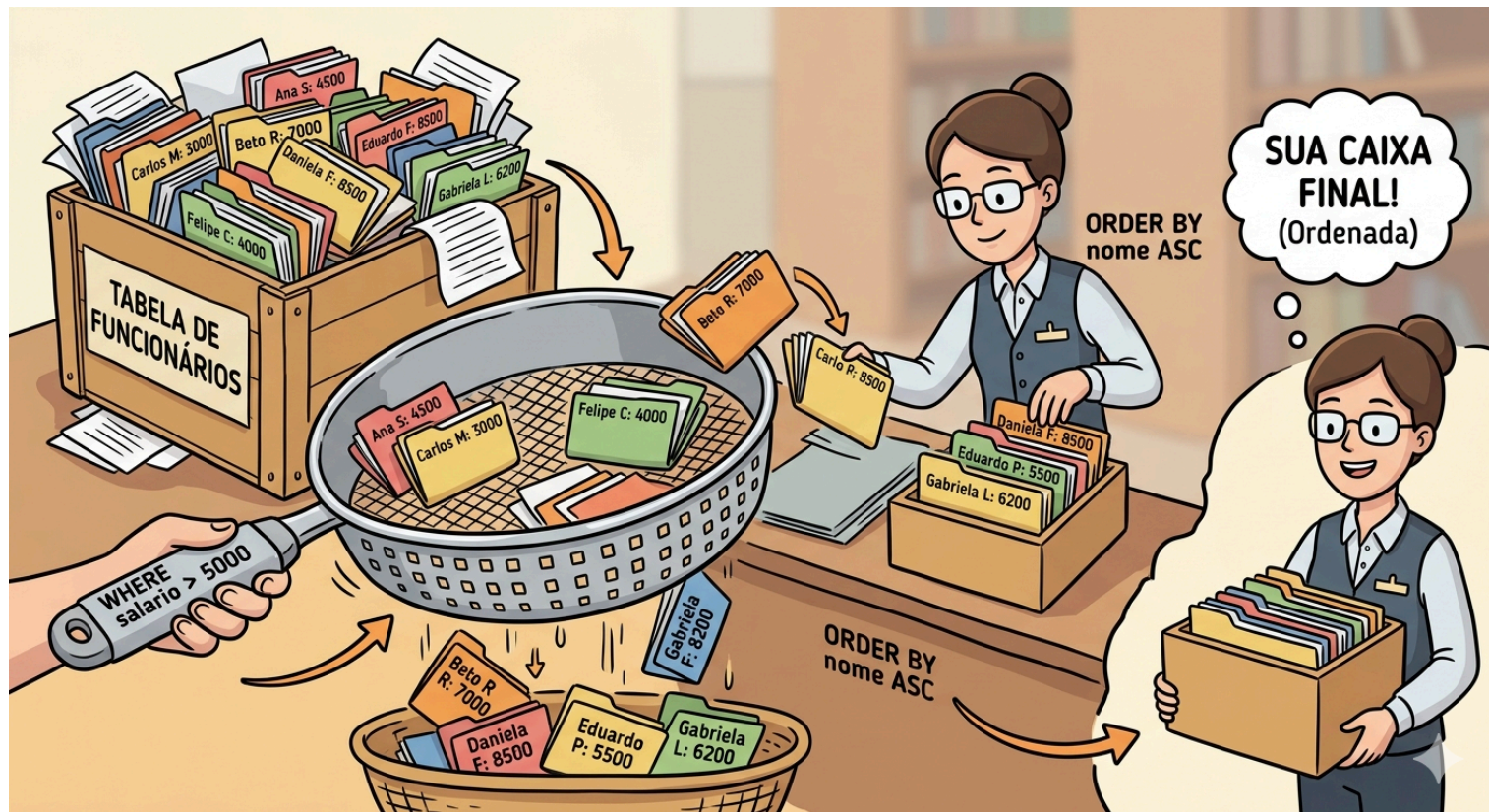
Filtragem Refinada e Ordenação (WHERE e ORDER BY)

A. CONCEITO

No dia a dia, raramente queremos ver *todos* os dados de uma tabela. A cláusula **WHERE** é o nosso filtro horizontal: ela avalia cada linha da tabela e retorna apenas aquelas em que a nossa condição for verdadeira (**TRUE**). Ela economiza processamento do servidor e rede, enviando para o front-end apenas o necessário. Além dos operadores básicos (**=**, **>**, **<**), utilizamos operadores de padrão de texto como o **LIKE** (para buscas parciais, excelente para barras de pesquisa) e o **BETWEEN** (para faixas de valores, como períodos de datas ou faixas salariais).

Após filtrar, geralmente precisamos apresentar a informação de forma organizada para o usuário final. É aí que entra o **ORDER BY**, responsável por ordenar o *Result Set* (conjunto de resultados) de forma ascendente (**ASC**) ou descendente (**DESC**).

B. VISUALIZANDO



C. EXEMPLO DE CÓDIGO

SQL

-- Seleccionamos as colunas desejadas

SELECT nome, cargo, salario, data_contratacao

FROM funcionarios

-- Filtramos: Salário entre 3k e 8k E que o nome comece com a letra 'A'

WHERE salario **BETWEEN** 3000.00 **AND** 8000.00

AND nome **LIKE** 'A%'

-- Ordenamos primeiro por cargo (alfabética) e depois por salário (maior pro menor)

ORDER BY cargo **ASC**, salario **DESC**;

D. EXPLICAÇÃO DO CÓDIGO

- **SELECT** e **FROM**: Definem o que queremos e de onde vem (projeção vertical).
- **WHERE** salario **BETWEEN**...: Em vez de usar `salario >= 3000 AND salario <= 8000`, o **BETWEEN** deixa o código mais limpo e legível.
- **LIKE** 'A%': O curinga **%** significa "qualquer quantidade de caracteres". Ou seja, obrigatoriamente a string começa com 'A' e termina com qualquer coisa (ex: Ana, Alberto).

- **ORDER BY:** A ordenação tem dois níveis. Se dois funcionários têm o mesmo cargo, o sistema vai olhar para a segunda

E. Dica de Performance: Quando for usar o **LIKE**, evite colocar o curinga no início da string (ex: **LIKE '%Silva'**). Isso impede o banco de dados de usar os índices (Índices B-Tree), obrigando-o a ler a tabela inteira (Full Table Scan), o que destrói a performance do sistema.

Agregação e Agrupamento (GROUP BY e HAVING)

A. CONCEITO

Até agora, olhamos para as linhas individualmente. Mas o mundo dos negócios precisa de resumos e totalizadores. As Funções de Agregação (**COUNT**, **SUM**, **AVG**, **MAX**, **MIN**) pegam múltiplas linhas e as transformam em um único valor (ex: o total de vendas do mês).

O verdadeiro poder surge quando combinamos essas funções com o **GROUP BY**. Ele agrupa linhas que têm os mesmos valores em colunas específicas (ex: agrupar clientes por estado). E se precisarmos filtrar esses *grupos resultantes*? Usamos a cláusula **HAVING**. Diferente do **WHERE** (que filtra linhas antes de agrupar), o **HAVING** filtra os grupos após a agregação.

B. VISUALIZANDO

ETAPA DO PROCESSO	DESCRIÇÃO E AÇÃO VISUAL	RESULTADO INTERMEDIÁRIO
FROM: Tabela de Vendas (Todos os Status)		Total Pastas: 10
WHERE: Filtra Linhas		10 Pastas
GROUP BY Vendedor		Montinhos: 10 Honitinhos: 20 Ventinhos: 35 Candedos: 10
SUM() Soma Valor		SUM() = 7000 => < F.SUM() = 8500
HAVING: Filtra Grupos	Beto R: = 7000 Daniela F: = 8500 Henrique D: = 9500 Gabriela L: = 6200 Isabela V: = 1500	5 grupos
ORDER BY Vendedor (ASC)		
RESULTADO FINAL		4 Vendedores

C. EXEMPLO DE CÓDIGO

```
SQL
-- Queremos saber o total gasto e a quantidade de compras por cliente
SELECT cliente_id,
       COUNT(pedido_id) AS total_pedidos,
       SUM(valor_total) AS valor_gasto
FROM pedidos
-- Filtramos apenas os pedidos entregues (atuando linha a linha)
WHERE status = 'Entregue'
-- Agrupamos os dados por cliente
```

GROUP BY cliente_id

-- Filtramos APENAS os clientes (grupos) que gastaram mais de mil reais

HAVING SUM(valor_total) > 1000.00;

D. EXPLICAÇÃO DO CÓDIGO

- COUNT() e SUM(): Estão calculando métricas em cima das linhas agrupadas. Usamos AS para criar um *alias* (apelido) e deixar a coluna do relatório mais apresentável.
- WHERE: Garantiu que pedidos cancelados ou em transporte não entrassem na soma.
- GROUP BY cliente_id: A regra de ouro é: se uma coluna está no SELECT e não tem função de agregação (como cliente_id), ela **obrigatoriamente** tem que estar no GROUP BY.
- HAVING: Verifica a soma de cada grupo e só exibe o grupo final se a condição for atingida.

E. Atenção: Um dos erros mais clássicos de desenvolvedores júniores é tentar usar funções de agregação no WHERE (ex: WHERE SUM(valor) > 100). O banco vai estourar um erro de sintaxe. Lembre-se: WHERE é para linhas, HAVING é para agregações.

Relacionando Entidades: A Arte das Junções (JOINS)

A. CONCEITO

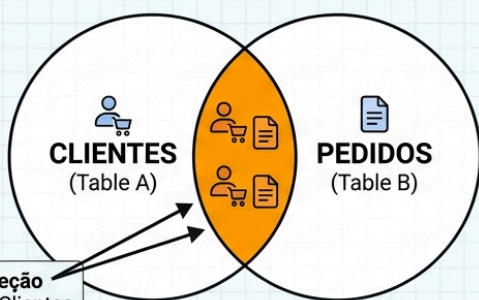
Na Fase de Normalização (aulas anteriores), nós separamos os dados em tabelas distintas para evitar anomalias (ex: tabela Clientes e tabela Pedidos). O JOIN é a operação que materializa o relacionamento entre essas entidades no momento da consulta, unindo as informações através de um atributo comum (geralmente ligando a Chave Primária - PK de uma tabela com a Chave Estrangeira - FK da outra).

Existem vários tipos de JOIN. O INNER JOIN é o mais comum: ele retorna apenas os registros que possuem correspondência exata em *ambas* as tabelas. Já os OUTER JOINS (como o LEFT JOIN) preservam os registros da tabela "da esquerda" mesmo que não encontrem correspondência na tabela "da direita", preenchendo os buracos com valores nulos (NULL).

B. VISUALIZANDO

Diagrama de Venn: INNER JOIN vs LEFT JOIN (Clientes & Pedidos)

INNER JOIN ($A \cap B$)

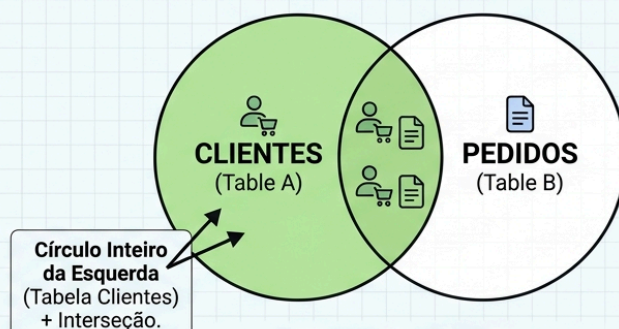


RESULTADO DO INNER JOIN

ID_Cliente	Nome	ID_Pedido	Valor
1	João	101	R\$ 150
2	Pedro	101	R\$ 150
3	Maria	102	R\$ 200
4	Ana	102	R\$ 200

Se um cliente não comprou nada, ele NÃO aparece.
Se um pedido não tem cliente válido, não aparece.

LEFT JOIN (A)



RESULTADO DO LEFT JOIN

ID_Cliente	Nome	ID_Pedido	Valor
1	João	101	R\$ 150
2	Pedro	NULL	NULL
3	Maria	102	R\$ 200
4	Ana	NULL	NULL

Traz TODOS os clientes. Se o cliente tiver pedido, traz os dados do pedido. Se o cliente nunca comprou, as colunas do pedido vêm como NULL.

C. EXEMPLO DE CÓDIGO

```
SQL
-- Selecionando dados de tabelas diferentes usando Aliases (c e p)
SELECT c.nome_cliente,
       c.email,
       p.data_pedido,
       p.valor_total
-- Tabela "da esquerda" (Base)
FROM clientes AS c
-- Junção interna com a Tabela "da direita"
INNER JOIN pedidos AS p
-- Condição de Junção (A ponte entre elas usando PK e FK)
ON c.cliente_id = p.cliente_id
-- Opcional: Podemos continuar filtrando normalmente
WHERE p.valor_total > 500.00;
```

D. EXPLICAÇÃO DO CÓDIGO

- **AS c** e **AS p**: Definir apelidos (alias) para as tabelas economiza muita digitação e deixa o código limpo.
- **SELECT c.nome_cliente**: Como estamos cruzando tabelas, é uma excelente prática prefaciar o nome da coluna com o alias da tabela para evitar ambiguidade (caso ambas tabelas tivessem colunas com o mesmo nome).
- **INNER JOIN ... ON ...**: A cláusula **ON** é onde a mágica acontece. É aqui que dizemos ao banco: "A chave primária **cliente_id** da tabela cliente deve ser igual à chave estrangeira **cliente_id** da tabela pedidos".

E. Dica de Soft Skill & Negócio: A escolha entre **INNER JOIN** e **LEFT JOIN** nunca é apenas técnica; é uma decisão de negócio. Se o Product Manager (PM) pede um "Relatório de Vendas", use **INNER JOIN**. Se ele pede um "Relatório de Clientes e suas Vendas", use **LEFT JOIN** para não ocultar clientes inativos e acabar mascarando os números da empresa.

FAQ - Perguntas Frequentes

Pergunta: Eu posso usar o **WHERE** e o **HAVING** na mesma query?

Resposta: Sim, e é muito comum! O **WHERE** filtra os dados brutos *antes* do agrupamento (economizando processamento), e o **HAVING** atua como um filtro final *depois* que os cálculos matemáticos do agrupamento já foram feitos.

Pergunta: Na prática real, usamos mais **INNER JOIN** ou **LEFT JOIN**?

Resposta: Depende do relatório, mas **LEFT JOIN** é criticamente útil para encontrar "inconsistências" ou "ausências" de ações. Por exemplo: "Liste todos os clientes (LEFT JOIN compras) ONDE a data da compra É NULA". Isso retorna exatamente os clientes que abandonaram a loja.

Pergunta: Posso usar o Alias criado no **SELECT** dentro do **WHERE**?

Resposta: Na maioria dos bancos relacionais (como PostgreSQL, SQL Server e MySQL), **não**. O banco de dados processa o **WHERE** *antes* do **SELECT**. Portanto, no momento de filtrar, o apelido ainda não "existe" para o servidor. Mas você pode usá-lo no **ORDER BY**!

Pergunta: Qual a diferença entre COUNT(*) e COUNT(coluna)?

Resposta: O COUNT(*) conta todas as linhas do seu resultado, indiscriminadamente. Já o COUNT(nome_da_coluna) conta apenas as linhas onde essa coluna específica **não é nula** (NOT NULL).

Pergunta: Fazer muitos JOINS deixa o sistema lento?

Resposta: Sim. Cruzar 10, 15 tabelas cria produtos cartesianos massivos. Por isso a importância da nossa aula anterior de Arquitetura e Modelagem. Um banco mal modelado exige muitos JOINS. Para contornar lentidão justificada, criamos Índices (INDEX) nas Chaves Estrangeiras, o que acelera a ponte absurdamente.

RESUMINDO

Checklist técnico da estrutura completa de uma consulta relacional:

- [x] **SELECT / FROM**: Definem quais colunas mostrar e a tabela origem principal.
 - [x] **JOIN ... ON**: Materializa relacionamentos cruzando chaves (PK e FK). **INNER** para interseção, **LEFT** para preservar a tabela base.
 - [x] **WHERE**: Filtro horizontal prévio linha a linha (**LIKE**, **BETWEEN**, **>**, **<**).
 - [x] **GROUP BY**: Aglutina registros com base em colunas de identidade, permitindo sumarizações.
 - [x] **Funções de Agregação**: Operações matemáticas em grupos (**SUM**, **AVG**, **MIN**, **MAX**, **COUNT**).
 - [x] **HAVING**: O "Where dos agrupamentos". Filtra apenas os resultados de agregações geradas pelo **GROUP BY**.
 - [x] **ORDER BY**: Entrega o *result set* final ao front-end ordenado visualmente.
-

EXERCÍCIOS

A lista de exercícios da aula será apresentada separado.

FECHAMENTO

Conexão com a Situação de Aprendizagem (SA):

Esta aula impacta diretamente na Matriz da sua Avaliação Prática (SA). Vocês entregarão o sistema completo. O "Dashboard" que vocês estão programando no front-end não servirá de nada se o backend não conseguir extrair os dados organizados do banco. As queries de relatório que usarão múltiplos cruzamentos (JOINS) e estatísticas (Agregações) são os motores de inteligência de negócios do projeto de vocês. Sem o que vimos hoje, a aplicação de vocês seria apenas um formulário de cadastro, não um sistema corporativo.